

Knowledge Representation

Lecture 8:

Resolution Principle

dr Anna Maria Radzikowska

Wydział Matematyki i Nauk Informatycznych

Pokój 227, gmach MiNI

E-mail: A.Radzikowska@mini.pw.edu.pl

Theorem proving

Recall...

Given a set T of sentences and a sentence α , a **formal proof of α from T** is the deduction system $DS = (\mathcal{L}, \mathcal{A}, \mathcal{R})$, written $T \vdash_{DS} \alpha$, is a sequence $(\alpha_0, \alpha_1, \dots, \alpha_n)$ of sentences such that

- $\alpha_0 \in \mathcal{A} \cup T$
- $\alpha_n = \alpha$
- for every $i = 1, \dots, n$, either $\alpha_i \in \mathcal{A} \cup T$ or α_i is a consequence of some reasoning rule applied to some sentences from $\alpha_0, \dots, \alpha_{i-1}$.

Theorem proving

Recall...

Given a set T of sentences and a sentence α , a **formal proof of α from T** is the deduction system $DS = (\mathcal{L}, \mathcal{A}, \mathcal{R})$, written $T \vdash_{DS} \alpha$, is a sequence $(\alpha_0, \alpha_1, \dots, \alpha_n)$ of sentences such that

- $\alpha_0 \in \mathcal{A} \cup T$
- $\alpha_n = \alpha$
- for every $i = 1, \dots, n$, either $\alpha_i \in \mathcal{A} \cup T$ or α_i is a consequence of some reasoning rule applied to some sentences from $\alpha_0, \dots, \alpha_{i-1}$.

The above definition does not give any procedure to determine a proof!

Automated theorem proving

Let T be a set of sentences (propositional or closed first-order formulas) and let β be a sentence.

The reasoning task:

Determine whether $T \vdash \beta$ or not.

We need some algorithm which allows us to solve this problem.

Automated theorem proving

Let T be a set of sentences (propositional or closed first-order formulas) and let β be a sentence.

The reasoning task:

Determine whether $T \vdash \beta$ or not.

We need some algorithm which allows us to solve this problem.

In propositional case the truth-table method is an example of such a technique.

In first-order case there is no analogous method!

Automated theorem proving

Let T be a set of sentences (propositional or closed first-order formulas) and let β be a sentence.

The reasoning task:

Determine whether $T \vdash \beta$ or not.

We need some algorithm which allows us to solve this problem.

In propositional case the truth-table method is an example of such a technique.

In first-order case there is no analogous method!

However, since FOPC is partly decidable, there is a method determining $T \vdash \beta$, provided that $T \models \beta$.

Knowledge base approach

Sometimes a set T of sentences (theory in logical terms) is identified with a *knowledge base* (KB) and a sentence β , which we aim to show it is a theorem of T , with a *query* (Q). Then

$T \vdash \beta$ — KB gives the answer **YES**

$T \not\vdash \beta$ — KB gives the answer **NO**.

Propositional Case

Recall...

Theorem 8.1

For any set $T = \{\alpha_1, \dots, \alpha_n\}$ of statements and for any (closed) formula β

- If $T = \{\alpha_1, \dots, \alpha_n\}$, then $T \models \beta$ iff $\models \alpha_1 \wedge \dots \wedge \alpha_n \rightarrow \beta$
- $T \models \beta$ iff $T \cup \{\neg\beta\}$ is unsatisfiable.

Recall...

Theorem 8.1

For any set $T = \{\alpha_1, \dots, \alpha_n\}$ of statements and for any (closed) formula β

- If $T = \{\alpha_1, \dots, \alpha_n\}$, then $T \models \beta$ iff $\models \alpha_1 \wedge \dots \wedge \alpha_n \rightarrow \beta$
- $T \models \beta$ iff $T \cup \{\neg\beta\}$ is unsatisfiable.

General idea:

- Add $\neg\beta$ to T and get $T' = T \cup \{\neg\beta\}$.
- Show that T' is inconsistent.

Literals and clauses

- A *literal* is an atom or its negation. Two literals λ_1 and λ_2 are *complementary* iff $\lambda_1 \Leftrightarrow \neg\lambda_2$

Literals and clauses

- A *literal* is an atom or its negation. Two literals λ_1 and λ_2 are *complementary* iff $\lambda_1 \Leftrightarrow \neg\lambda_2$
- A *clause* is a formula of the form:

$$\boxed{\gamma = \lambda_1 \vee \dots \vee \lambda_n} \quad n \geq 0$$

where λ_i are literals.

If $n = 0$, then we have the empty clause denoted by $\square$ (*falsity*).

Resolution rule

- Let λ be an atom and let α and β be clauses.

A **resolution rule** (RR) is the reasoning rule of the form

$$\frac{\lambda \vee \alpha, \neg\lambda \vee \beta}{\alpha \vee \beta}$$

Resolution rule

- Let λ be an atom and let α and β be clauses.

A **resolution rule** (RR) is the reasoning rule of the form

$$\frac{\lambda \vee \alpha, \neg\lambda \vee \beta}{\alpha \vee \beta}$$

- The consequent $\alpha \vee \beta$ of RR is called a **resolvent** of $\gamma_1 = \lambda \vee \alpha$ and $\gamma_2 = \neg\lambda \vee \beta$, in symbols $Res(\gamma_1, \gamma_2)$.

Resolution rule

- Let λ be an atom and let α and β be clauses.

A **resolution rule** (RR) is the reasoning rule of the form

$$\boxed{\frac{\lambda \vee \alpha, \neg\lambda \vee \beta}{\alpha \vee \beta}}$$

- The consequent $\alpha \vee \beta$ of RR is called a **resolvent** of $\gamma_1 = \lambda \vee \alpha$ and $\gamma_2 = \neg\lambda \vee \beta$, in symbols $Res(\gamma_1, \gamma_2)$.
- Note that $\{\lambda \vee \alpha, \neg\lambda \vee \beta\} \models \alpha \vee \beta$.

Resolution rule

- Let λ be an atom and let α and β be clauses.

A **resolution rule** (RR) is the reasoning rule of the form

$$\boxed{\frac{\lambda \vee \alpha, \neg\lambda \vee \beta}{\alpha \vee \beta}}$$

- The consequent $\alpha \vee \beta$ of RR is called a **resolvent** of $\gamma_1 = \lambda \vee \alpha$ and $\gamma_2 = \neg\lambda \vee \beta$, in symbols $Res(\gamma_1, \gamma_2)$.
- Note that $\{\lambda \vee \alpha, \neg\lambda \vee \beta\} \models \alpha \vee \beta$.
- RR is **applicable** for two clauses γ_1 and γ_2 iff γ_1 contains a literal λ complementary to some literal λ' in γ_2 .

Resolution rule

- Let λ be an atom and let α and β be clauses.

A **resolution rule** (RR) is the reasoning rule of the form

$$\boxed{\frac{\lambda \vee \alpha, \neg\lambda \vee \beta}{\alpha \vee \beta}}$$

- The consequent $\alpha \vee \beta$ of RR is called a **resolvent** of $\gamma_1 = \lambda \vee \alpha$ and $\gamma_2 = \neg\lambda \vee \beta$, in symbols $Res(\gamma_1, \gamma_2)$.
- Note that $\{\lambda \vee \alpha, \neg\lambda \vee \beta\} \models \alpha \vee \beta$.
- RR is **applicable** for two clauses γ_1 and γ_2 iff γ_1 contains a literal λ complementary to some literal λ' in γ_2 .
- To apply RR we need a special form of T and $\neg\beta$.

Conjunctive Normal Form

A formula α is in the *Conjunctive Normal Form* (*CNF*) iff

$$\alpha = \bigwedge_{i=1}^n \bigvee_{j=1}^{k(i)} \lambda_{ij}$$

where λ_{ij} are literals for all $i = 1, \dots, n$, $j = 1, \dots, k(i)$.

Note: α is in the CNF iff it is conjunction of clauses. It can be viewed as a set of the respective clauses.

Conjunctive Normal Form

A formula α is in the **Conjunctive Normal Form** (**CNF**) iff

$$\alpha = \bigwedge_{i=1}^n \bigvee_{j=1}^{k(i)} \lambda_{ij}$$

where λ_{ij} are literals for all $i = 1, \dots, n$, $j = 1, \dots, k(i)$.

Note: α is in the CNF iff it is conjunction of clauses. It can be viewed as a set of the respective clauses.

Theorem 8.2 A formula α can be transformed to the formula α' in CNF such that $\alpha \Leftrightarrow \alpha'$.

Translation to CNF

Step 1. Eliminate $\rightarrow$ and $\equiv$. Use:

$$\alpha \rightarrow \beta \quad \rightsquigarrow \quad \neg\alpha \vee \beta$$

$$\alpha \equiv \beta \quad \rightsquigarrow \quad (\neg\alpha \vee \beta) \wedge (\neg\beta \vee \alpha)$$

Step 2. Move $\neg$ to the right.

$$\neg(\alpha \wedge \beta) \quad \rightsquigarrow \quad \neg\alpha \vee \neg\beta$$

$$\neg(\alpha \vee \beta) \quad \rightsquigarrow \quad \neg\alpha \wedge \neg\beta$$

$$\neg\neg\alpha \quad \rightsquigarrow \quad \alpha$$

Step 3. Distributivity.

$$\alpha \vee (\beta \wedge \gamma) \quad \rightsquigarrow \quad (\alpha \vee \beta) \wedge (\alpha \vee \gamma).$$

Resolution principle (cont.)

Task: Show that $T \vdash \beta$.

Procedure:

- Step 1.** Transform every $\alpha \in T$ and $\neg\beta$ into CNF. Get the set $CL = CL(T, \neg\beta)$ of clauses.
- Step 2.** If there are two clauses $\gamma_1, \gamma_2 \in CL$ for which RR is applicable, apply RR and obtain $\gamma = Res(\gamma_1, \gamma_2)$. Otherwise STOP – return **NO** and exit.
- Step 3.** If $\gamma = \square$ then STOP — return **YES** and exit.
- Step 4.** Put $CL := CL \cup \{\gamma\}$ and go to **Step 2**.

Resolution proof

Let T be a set of propositional formulas and let β be a propositional formula. A **resolution proof** of β from T is a sequence $(\gamma_0, \gamma_1, \dots, \gamma_n)$ of clauses such that

- $\gamma_0 \in CL(T, \neg\beta)$
- $\gamma_n = \square$
- for $i \geq 1$, either $\gamma_i \in CL(T, \neg\beta)$ or $\gamma_i = Res(\gamma_j, \gamma_k)$ for some $j, k < i$

Resolution proof

Let T be a set of propositional formulas and let β be a propositional formula. A **resolution proof** of β from T is a sequence $(\gamma_0, \gamma_1, \dots, \gamma_n)$ of clauses such that

- $\gamma_0 \in CL(T, \neg\beta)$
- $\gamma_n = \square$
- for $i \geq 1$, either $\gamma_i \in CL(T, \neg\beta)$ or $\gamma_i = Res(\gamma_j, \gamma_k)$ for some $j, k < i$

Theorem 8.3 (Soundness and completeness) $T \vdash \beta$ iff there is a resolution proof of β from T .

Resolution tree

A **resolution tree** for T and β is a binary tree such that

- each node is labelled by a clause
- each leaf is labelled by some clause from $CL(T, \neg\beta)$
- if v is labelled by γ , v_1 is labelled by γ_1 , v_2 is labelled by γ_2 , and v is the father of v_1 and v_2 , then $\gamma = Res(\gamma_1, \gamma_2)$
- If $T \vdash \beta$ then $\square$ is the root of the tree.

Example 8.1

Let T be

$$\alpha_1 = \textit{bird}$$

$$\alpha_2 = \textit{bird} \rightarrow \textit{flies}$$

$$\alpha_2 = \textit{flies} \rightarrow \textit{hasWings}$$

and $\beta = \textit{hasWings}$.

Transformation of T and $\neg\beta$ into CNF:

$$\alpha'_1 = \textit{bird}$$

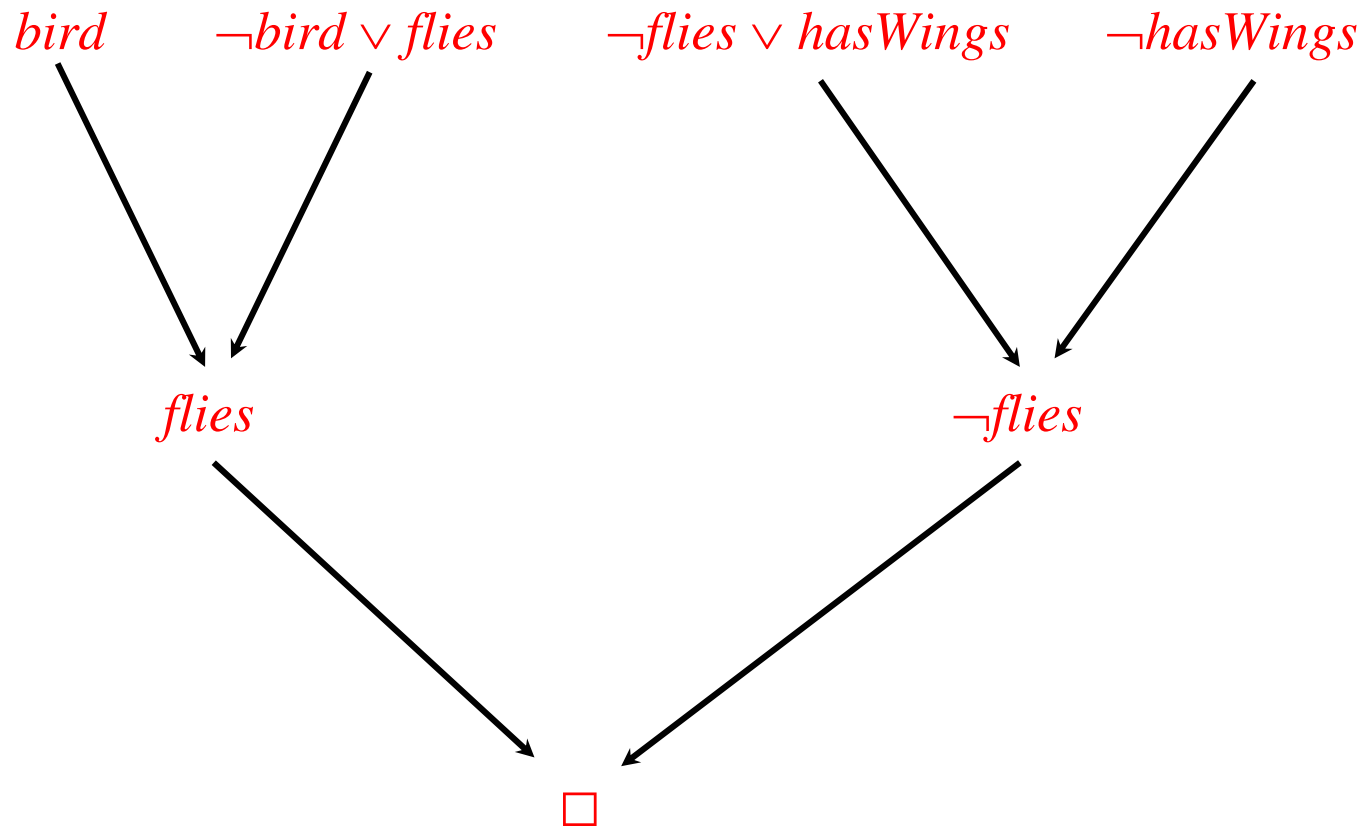
$$\alpha'_2 = \neg\textit{bird} \vee \textit{flies}$$

$$\alpha'_3 = \neg\textit{flies} \vee \textit{hasWings}$$

$$\beta' = \textit{hasWings}.$$

Example 8.1 (cont.)

Resolution tree:



Resolution principle – remarks

- In general, the resolution method is not effective — too many clauses can be generated.

Resolution principle – remarks

- In general, the resolution method is not effective — too many clauses can be generated.
- In order to increase efficiency, some constraints should be imposed on the selection of clauses that are to be resolved at a point of time.

Resolution principle – remarks

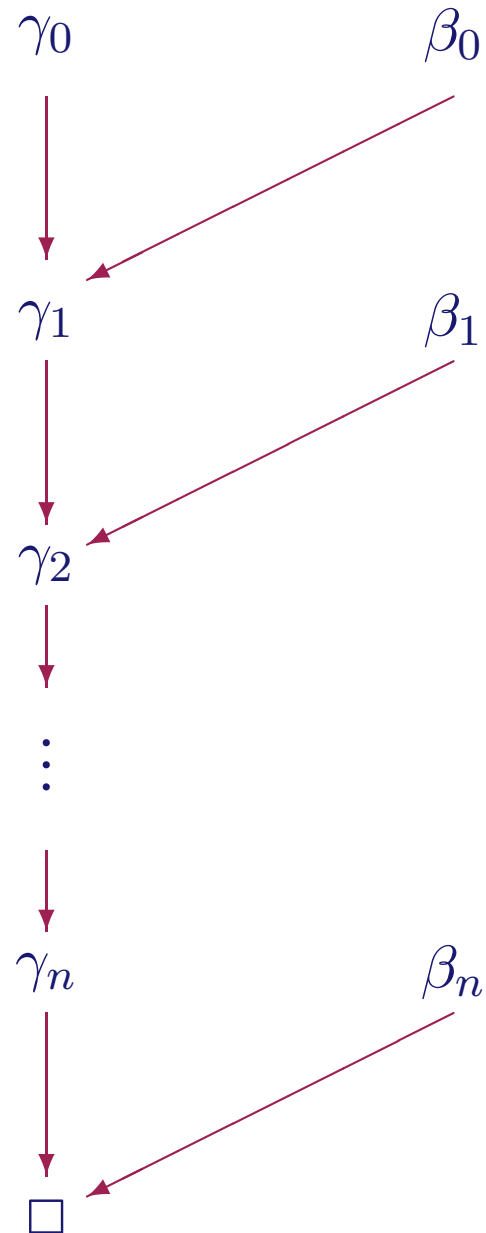
- In general, the resolution method is not effective — too many clauses can be generated.
- In order to increase efficiency, some constraints should be imposed on the selection of clauses that are to be resolved at a point of time.
- *Linear resolution* is based on this idea.

Linear resolution

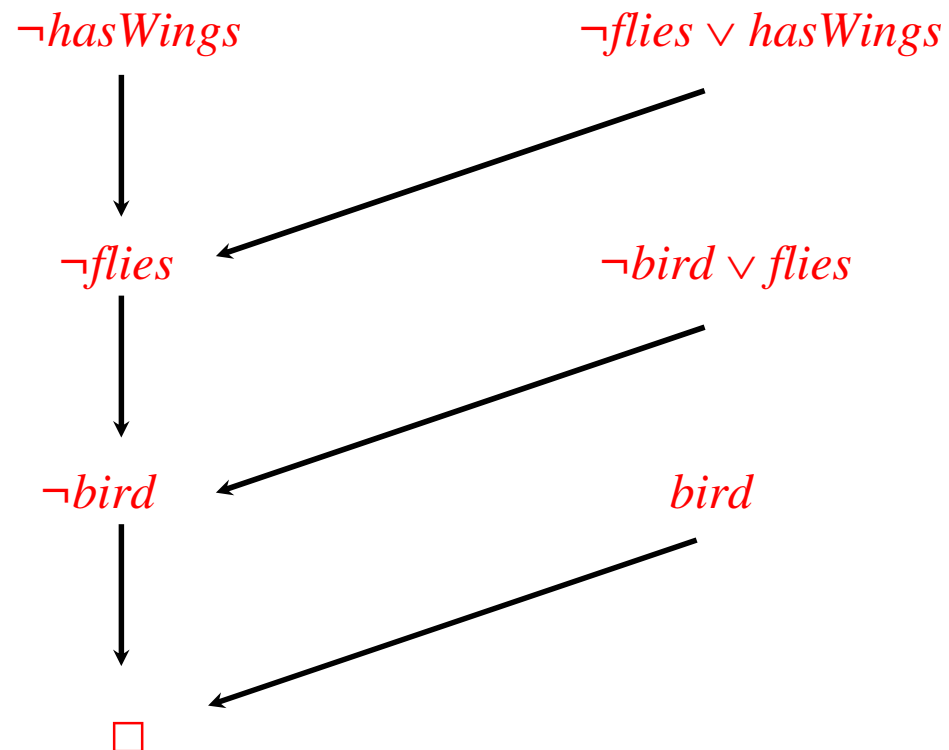
A **linear resolution proof** of β from T is a pair (G, B) of finite sequences of clauses such that

- $G = (\gamma_0, \dots, \gamma_n, \square)$ (**main clauses**),
 $B = (\beta_0, \dots, \beta_n)$ (**side clauses**)
- $\gamma_0 \in CL(\neg\beta)$ is called a **top clause**
- $\beta_0 \in CL(T, \neg\beta)$ and for each $i = 1, \dots, n$,
 $\beta_i \in CL(T, \neg\beta) \cup \{\gamma_0, \dots, \gamma_{i-1}\}$
- for each $i = 1, \dots, n$, $\gamma_i = Res(\gamma_{i-1}, \beta_{i-1})$.

Linear resolution (cont.)



Example 8.1 (cont.)



First-order case

What do we need?

In order to show $T \vdash \beta$ using RR we need:

- Transform each sentence $\alpha \in T$ and $\neg\beta$ into the set of clauses.
What to do with quantifiers?

What do we need?

In order to show $T \vdash \beta$ using RR we need:

- Transform each sentence $\alpha \in T$ and $\neg\beta$ into the set of clauses.
What to do with quantifiers?
- Unification of literals.
E.g., $Student(x)$ and $Student(Peter)$ are NOT the same literals!

What do we need?

In order to show $T \vdash \beta$ using RR we need:

- Transform each sentence $\alpha \in T$ and $\neg\beta$ into the set of clauses.
What to do with quantifiers?
- Unification of literals.
E.g., $Student(x)$ and $Student(Peter)$ are NOT the same literals!
- Some additional techniques will be also useful.

Prenex Conjunctive Normal Form

A (first-order) formula α is in the **PCNF** iff

$$\alpha = Q_1 x_1 \ Q_2 x_2 \ \dots \ Q_m x_m \cdot \bigwedge_{i=1}^n \bigvee_{j=1}^{k(i)} \lambda_{ij}$$

where

- $Q_1, \dots, Q_m$ are quantifiers
- $x_1, \dots, x_m$ are distinct variables
- λ_{ij} are literals.

Prefix — $Q_1 x_1 \ Q_2 x_2 \ \dots \ Q_m x_m$

Matrix — $\bigwedge_{i=1}^n \bigvee_{j=1}^{k(i)} \lambda_{ij}.$

Prenex Conjunctive Normal Form

A (first-order) formula α is in the **PCNF** iff

$$\alpha = Q_1 x_1 \ Q_2 x_2 \ \dots \ Q_m x_m \cdot \bigwedge_{i=1}^n \bigvee_{j=1}^{k(i)} \lambda_{ij}$$

where

- $Q_1, \dots, Q_m$ are quantifiers
- $x_1, \dots, x_m$ are distinct variables
- λ_{ij} are literals.

Prefix — $Q_1 x_1 \ Q_2 x_2 \ \dots \ Q_m x_m$

Matrix — $\bigwedge_{i=1}^n \bigvee_{j=1}^{k(i)} \lambda_{ij}.$

Theorem 8.4 For every formula α there exists a formula α' in PCNF such that $\alpha \Leftrightarrow \alpha'$.

Translation to PCNF — Algorithm

Input: a formula α

Output: PCNF of α .

Translation to PCNF — Algorithm

Input: a formula α

Output: PCNF of α .

Algorithm:

Step 1. Eliminate redundant quantifiers.

Eliminate in α any $\forall x$ (resp. $\exists x$), which scope does not contain x .

Denote the resulting formula by α_1 .

Translation to PCNF — Algorithm

Input: a formula α

Output: PCNF of α .

Algorithm:

Step 1. Eliminate redundant quantifiers.

Eliminate in α any $\forall x$ (resp. $\exists x$), which scope does not contain x .

Denote the resulting formula by α_1 .

E.g., $\forall x \exists y. P(x) \rightarrow P(a, x)$, $\exists y$ is redundant.

Translation to PCNF (cont.)

Step 2. Rename variables.

Repeat

- Take the leftmost subformula of α_1 of the form $\forall x. \phi(x)$ (resp. $\exists x. \phi(x)$) such that x occurs in other part of α_1 .
- Replace this subformula by $\forall y. \phi(y)$ (resp. $\exists y. \phi(x)$), where y is a **new** variable.

until all quantified variables are different and no variable is both bound and free. Denote the resulting formula by α_2 .

Translation to PCNF (cont.)

Step 2. Rename variables.

Repeat

- Take the leftmost subformula of α_1 of the form $\forall x. \phi(x)$ (resp. $\exists x. \phi(x)$) such that x occurs in other part of α_1 .
- Replace this subformula by $\forall y. \phi(y)$ (resp. $\exists y. \phi(x)$), where y is a **new** variable.

until all quantified variables are different and no variable is both bound and free. Denote the resulting formula by α_2 .

E.g., $\alpha_1 = (\forall x. A(x) \rightarrow B(x)) \wedge (\exists x. C(x))$

$\alpha_2 = (\forall x. A(x) \rightarrow B(x)) \wedge (\exists y. C(y)).$

Translation to PCNF (cont.)

Step 3. Eliminate $\neg$ and $\equiv$.

$$\begin{aligned}\beta \rightarrow \phi &\rightsquigarrow \neg\beta \vee \phi \\ \beta \equiv \phi &\rightsquigarrow (\neg\beta \vee \phi) \wedge (\neg\phi \vee \beta)\end{aligned}$$

Denote the resulting formula by α_3 .

Translation to PCNF (cont.)

Step 3. Eliminate $\neg$ and $\equiv$.

$$\begin{aligned}\beta \rightarrow \phi &\sim \neg\beta \vee \phi \\ \beta \equiv \phi &\sim (\neg\beta \vee \phi) \wedge (\neg\phi \vee \beta)\end{aligned}$$

Denote the resulting formula by α_3 .

Step 4. Move $\neg$ inward.

$$\begin{aligned}\neg\forall x \beta(x) &\sim \exists x \neg\beta(x) \\ \neg\exists x \beta(x) &\sim \forall x \neg\beta(x) \\ \neg(\beta \vee \phi) &\sim \neg\beta \wedge \neg\phi \\ \neg(\beta \wedge \phi) &\sim \neg\beta \vee \neg\phi \\ \neg\neg\beta &\sim \beta.\end{aligned}$$

Denote the resulting formula by α_4 .

Translation to PCNF (cont.)

Step 5. Move quantifiers to the left.

$$\begin{array}{lll} (\forall x \beta) \wedge \phi & \rightsquigarrow & \forall x (\beta \wedge \phi) \\ \phi \wedge (\forall x \beta) & \rightsquigarrow & \forall x (\phi \wedge \beta) \\ (\forall x \beta) \vee \phi & \rightsquigarrow & \forall x (\beta \vee \phi) \\ \phi \vee (\forall x \beta) & \rightsquigarrow & \forall x (\phi \vee \beta) \\ (\exists x \beta) \wedge \phi & \rightsquigarrow & \exists x (\beta \wedge \phi) \\ \phi \wedge (\exists x \beta) & \rightsquigarrow & \exists x (\phi \wedge \beta) \\ (\exists x \beta) \vee \phi & \rightsquigarrow & \exists x (\beta \vee \phi) \\ \phi \vee (\exists x \beta) & \rightsquigarrow & \exists x (\phi \vee \beta). \end{array}$$

Denote the resulting formula by α_5 .

Translation to PCNF (cont.)

Step 6. Distribution.

$$\begin{aligned}(\beta \wedge \phi) \vee \psi &\rightsquigarrow (\beta \vee \psi) \wedge (\phi \vee \psi) \\ \beta \vee (\phi \wedge \psi) &\rightsquigarrow (\beta \vee \phi) \wedge (\beta \vee \psi).\end{aligned}$$

Denote the resulting formula by α_6 .

Translation to PCNF (cont.)

Step 6. Distribution.

$$\begin{aligned}(\beta \wedge \phi) \vee \psi &\rightsquigarrow (\beta \vee \psi) \wedge (\phi \vee \psi) \\ \beta \vee (\phi \wedge \psi) &\rightsquigarrow (\beta \vee \phi) \wedge (\beta \vee \psi).\end{aligned}$$

Denote the resulting formula by α_6 .

Theorem 8.5 The resulting formula α_6 is in PCNF and $\alpha \Leftrightarrow \alpha_6$.

Example 8.2

Consider a formula $\alpha = P(x, y) \rightarrow (\exists x. Q(x) \wedge \exists z. R(x, y))$.

$$\alpha_1 = P(x, y) \rightarrow (\exists x. Q(x) \wedge R(x, y))$$

$$\alpha_2 = P(u, y) \rightarrow (\exists x. Q(x) \wedge R(x, y))$$

$$\alpha_3 = \neg P(u, y) \vee (\exists x. Q(x) \wedge (x, y))$$

$$\alpha_4 = \alpha_3$$

$$\alpha_5 = \exists x. (\neg P(u, y) \vee (Q(x) \wedge R(x, y)))$$

$$\alpha_6 = \exists x. ((\neg P(u, y) \vee Q(x)) \wedge (\neg P(u, y) \vee R(x, y))).$$

Clauses

Assume that α is in PCNF, i.e. $\alpha = Q_1 x_1 \dots Q_n x_n \cdot \bigwedge_{i=1}^m \bigvee_{j=1}^{k(i)} \lambda_{ij}$.
Consider the following two cases:

- All Q_i are $\forall$ — α can be identified with the set CL of m clauses:

$$CL = \{\gamma_1(x), \dots, \gamma_m(x)\},$$

where $x = (x_1 \dots x_n)$ and $\gamma_i = \bigvee_{j=1}^{k(i)} \lambda_{ij}(x)$.

Clauses

Assume that α is in PCNF, i.e. $\alpha = Q_1 x_1 \dots Q_n x_n \cdot \bigwedge_{i=1}^m \bigvee_{j=1}^{k(i)} \lambda_{ij}$.
Consider the following two cases:

- All Q_i are $\forall$ — α can be identified with the set CL of m clauses:

$$CL = \{\gamma_1(x), \dots, \gamma_m(x)\},$$

where $x = (x_1 \dots x_n)$ and $\gamma_i = \bigvee_{j=1}^{k(i)} \lambda_{ij}(x)$.

- For some $i = 1, \dots, n$, $Q_i = \exists$. We need a method to eliminate existential quantifiers. This process is called a **Skolemization**.

Skolemization

Skolemization is a procedure of elimination of existential quantifiers.

Skolemization

Skolemization is a procedure of elimination of existential quantifiers.

Assume that α is in PCNF, i.e. $\alpha = Q_1x_1 \dots Q_nx_m \cdot \phi$.

Skolemization

Skolemization is a procedure of elimination of existential quantifiers.

Assume that α is in PCNF, i.e. $\alpha = Q_1x_1 \dots Q_nx_m. \phi$.

- Let Q_k be the leftmost existential quantifier
 - If $k = 1$ then substitute all occurrence of x_k in ϕ by a new constant a (**Skolem constant**) and remove $\exists x_1$ from the prefix.
 - If $k > 1$ then substitute all occurrences of x_k in ϕ by a new function $f(x_1, \dots, x_{k-1})$ (**Skolem function**) and remove $\exists x_k$ from the prefix.
- Repeat the process until all $\exists$ are eliminated.

Skolemization (cont.)

Example 8.3

• $\alpha = \exists x \forall y. Teaches(x, y).$

After Skolemization:

$\alpha' = \forall y. Teaches(a, y),$ where a is a Skolem constant.

Skolemization (cont.)

Example 8.3

• $\alpha = \exists x \forall y. Teaches(x, y).$

After Skolemization:

$$\alpha' = \forall y. Teaches(a, y), \text{ where } a \text{ is a Skolem constant.}$$

• $\beta = \forall x \exists y. Likes(x, y).$

After Skolemization:

$$\beta' = \forall x. Likes(x, f(x)), \text{ where } f \text{ is a Skolem function.}$$

Substitution

Assume that the following two clauses are given:

$$\gamma_1 = \neg Student(x) \vee Adult(x)$$

$$\gamma_2 = Student(Peter).$$

Note that γ_1 corresponds to the formula

$$\forall x. Student(x) \rightarrow Adult(x),$$

so γ_1 and γ_2 imply $Adult(Peter)$.

However, RR is not applicable to γ_1 and γ_2 .

Substitution

Assume that the following two clauses are given:

$$\gamma_1 = \neg Student(x) \vee Adult(x)$$

$$\gamma_2 = Student(Peter).$$

Note that γ_1 corresponds to the formula

$$\forall x. Student(x) \rightarrow Adult(x),$$

so γ_1 and γ_2 imply $Adult(Peter)$.

However, RR is not applicable to γ_1 and γ_2 .

We need a **substitution**, say $x \leftarrow Peter$.

Substitution (cont.)

A **substitution** Θ is a set

$$\Theta = \{x_1 \leftarrow t_1, \dots, x_n \leftarrow t_n\}$$

such that

- $x_1, \dots, x_n$ are distinct variables
- $t_1, \dots, t_n$ are terms not containing any x_i , $i = 1, \dots, n$.

An empty substitution is denoted by ε .

Substitution (cont.)

A **substitution** Θ is a set

$$\Theta = \{x_1 \leftarrow t_1, \dots, x_n \leftarrow t_n\}$$

such that

- $x_1, \dots, x_n$ are distinct variables
- $t_1, \dots, t_n$ are terms not containing any x_i , $i = 1, \dots, n$.

An empty substitution is denoted by ε .

Example 8.4

- $\Theta_1 = \{x \leftarrow a, y \leftarrow f(z)\}$ is a substitution
- $\Theta_2 = \{x \leftarrow a, y \leftarrow f(x)\}$ is **NOT** a substitution.

Substitution (cont.)

A **substitution** Θ is a set

$$\Theta = \{x_1 \leftarrow t_1, \dots, x_n \leftarrow t_n\}$$

such that

- $x_1, \dots, x_n$ are distinct variables
- $t_1, \dots, t_n$ are terms not containing any x_i , $i = 1, \dots, n$.

An empty substitution is denoted by ε .

Example 8.4

- $\Theta_1 = \{x \leftarrow a, y \leftarrow f(z)\}$ is a substitution
- $\Theta_2 = \{x \leftarrow a, y \leftarrow f(x)\}$ is **NOT** a substitution.

Substitutions are applied to formulas and terms.

Application of substitutions

Let

- e – an expression (either a term or a formula)
- $\Theta = \{x_1 \leftarrow t_1, \dots, x_n \leftarrow t_n\}$ – a substitution.

An application of Θ to e , written $e\Theta$, is the expression e' such the all occurrences of $x_1, \dots, x_n$ in e are respectively replaced by $t_1, \dots, t_n$.

Application of substitutions

Let

- e – an expression (either a term or a formula)
- $\Theta = \{x_1 \leftarrow t_1, \dots, x_n \leftarrow t_n\}$ – a substitution.

An application of Θ to e , written $e\Theta$, is the expression e' such the all occurrences of $x_1, \dots, x_n$ in e are respectively replaced by $t_1, \dots, t_n$.

Example 8.5

- $e = \neg P(x, y) \vee Q(x, f(x))$
- $\Theta = \{x \leftarrow a, y \leftarrow g(z)\}$
- $e\Theta = \neg P(a, g(z)) \vee Q(a, f(a))$.

Composition of substitutions

Given two substitutions

- $\Theta_1 = \{x_1 \leftarrow t_1, \dots, x_n \leftarrow t_n\}$
- $\Theta_2 = \{y_1 \leftarrow v_1, \dots, y_k \leftarrow v_k\},$

the **composition** $\Theta_1\Theta_2$ is the set obtained from

$$\{x_1 \leftarrow t_1\Theta_2, \dots, x_n \leftarrow t_n\Theta_2, y_1 \leftarrow v_1, \dots, y_k \leftarrow v_k\}$$

by removing expressions of the form

- $x_i \leftarrow t_i\Theta_2$ such that $x_i = t_i\Theta_2$
- $y_i \leftarrow v_i$ such that $y_i \in \{x_1, \dots, x_n\}.$

Composition of substitutions

Given two substitutions

- $\Theta_1 = \{x_1 \leftarrow t_1, \dots, x_n \leftarrow t_n\}$

- $\Theta_2 = \{y_1 \leftarrow v_1, \dots, y_k \leftarrow v_k\},$

the **composition** $\Theta_1 \Theta_2$ is the set obtained from

$$\{x_1 \leftarrow t_1 \Theta_2, \dots, x_n \leftarrow t_n \Theta_2, y_1 \leftarrow v_1, \dots, y_k \leftarrow v_k\}$$

by removing expressions of the form

- $x_i \leftarrow t_i \Theta_2$ such that $x_i = t_i \Theta_2$

- $y_i \leftarrow v_i$ such that $y_i \in \{x_1, \dots, x_n\}.$

Example 8.6

$$\Theta_1 = \{x \leftarrow g(y), z \leftarrow y\}$$

$$\Theta_2 = \{x \leftarrow a, y \leftarrow b, z \leftarrow f(c)\}$$

$$\Theta_1 \Theta_2 = \{x \leftarrow g(b), z \leftarrow b, y \leftarrow b\}.$$

Unifiers

- A set $Ex = \{e_1, \dots, e_n\}$ of expressions is **unifiable** iff there is a substitution Θ such that $e_1\Theta = e_2\Theta = \dots = e_n\Theta$.

Unifiers

- A set $Ex = \{e_1, \dots, e_n\}$ of expressions is **unifiable** iff there is a substitution Θ such that $e_1\Theta = e_2\Theta = \dots = e_n\Theta$.
- If Ex is unifiable then any substitution which makes it unifiable is called a **unifier** of Ex .

Unifiers

- A set $Ex = \{e_1, \dots, e_n\}$ of expressions is **unifiable** iff there is a substitution Θ such that $e_1\Theta = e_2\Theta = \dots = e_n\Theta$.
- If Ex is unifiable then any substitution which makes it unifiable is called a **unifier** of Ex .
- For example, for $Ex = \{P(x), P(y)\}$ the following substitutions are unifiers of Ex :

$$\Theta_1 = \{x \leftarrow y\}$$

$$\Theta_2 = \{y \leftarrow x\}$$

$$\Theta_3 = \{x \leftarrow a, y \leftarrow a\}$$

$$\Theta_4 = \{x \leftarrow f(z), y \leftarrow f(z)\}.$$

Most General Unifier

Let Ex be a unifiable set of expressions. A unifier Θ of Ex is called the **most general unifier**, in symbols $MGU(Ex)$, iff for every unifier θ of Ex there is a substitution δ such that $\theta = \Theta\delta$.

Most General Unifier

Let Ex be a unifiable set of expressions. A unifier Θ of Ex is called the **most general unifier**, in symbols $MGU(Ex)$, iff for every unifier θ of Ex there is a substitution δ such that $\theta = \Theta\delta$.

Example 8.7 For $Ex = \{P(x), P(y)\}$, there are two MGUs:

$$\Theta_1 = \{x \leftarrow y\}$$

$$\Theta_2 = \{y \leftarrow x\}$$

since

$$\Theta_3 = \{x \leftarrow a, y \leftarrow a\} = \Theta_1\{y \leftarrow a\}$$

$$\Theta_4 = \{x \leftarrow f(z), y \leftarrow f(z)\} = \Theta_2\{x \leftarrow f(z)\}.$$

Most General Unifier

Let Ex be a unifiable set of expressions. A unifier Θ of Ex is called the **most general unifier**, in symbols $MGU(Ex)$, iff for every unifier θ of Ex there is a substitution δ such that $\theta = \Theta\delta$.

Example 8.7 For $Ex = \{P(x), P(y)\}$, there are two MGUs:

$$\Theta_1 = \{x \leftarrow y\}$$

$$\Theta_2 = \{y \leftarrow x\}$$

since

$$\Theta_3 = \{x \leftarrow a, y \leftarrow a\} = \Theta_1\{y \leftarrow a\}$$

$$\Theta_4 = \{x \leftarrow f(z), y \leftarrow f(z)\} = \Theta_2\{x \leftarrow f(z)\}.$$

Every unifiable set Ex of expressions has at least one MGU.

Disagreement sets

How to determine $MGU(Ex)$ if it exists?

Disagreement sets

How to determine $MGU(Ex)$ if it exists?

$Ex = \{e_1, \dots, e_n\}$ — set of expressions.

A **disagreement set** of Ex is the set $DS(Ex) = \{e'_1, \dots, e'_m\}$ of expressions defined as:

- Consider $e_1, \dots, e_n$ as a string of symbols.
- Determine the leftmost position k at which at least two expressions from Ex have different symbols.
- For each $i = 1, \dots, k$, put e'_i to be the subexpression of e_i starting from the position k .

Disagreement sets

How to determine $MGU(Ex)$ if it exists?

$Ex = \{e_1, \dots, e_n\}$ — set of expressions.

A **disagreement set** of Ex is the set $DS(Ex) = \{e'_1, \dots, e'_m\}$ of expressions defined as:

- Consider $e_1, \dots, e_n$ as a string of symbols.
- Determine the leftmost position k at which at least two expressions from Ex have different symbols.
- For each $i = 1, \dots, k$, put e'_i to be the subexpression of e_i starting from the position k .

Example 8.8

$$Ex = \{P(x, f(a)), P(x, c), P(f(y), d)\}$$
$$DS(Ex) = \{x, f(y)\}.$$

Unification algorithm

Input: a set Ex of expressions.

Output: **YES** iff Ex is unifiable (and $MGU(Ex)$); otherwise **NO**.

Step 1: Set $k = 0$, $E_0 = Ex$, $\Theta_0 = \varepsilon$

Step 2: If E_k is a singleton then STOP — Ex is unifiable and $MGU(Ex) = \Theta_k$; otherwise go to **Step 3**.

Step 3: Determine $DS(E_k)$.

Step 4: If $DS(E_k)$ contains a variable x and a term t not containing x , put

$$\Theta_{k+1} = \Theta_k \{x \leftarrow t\}$$

$$E_{k+1} = Ex_k \Theta_{k+1}$$

$$k := k + 1$$

go to **Step 2**

otherwise STOP — Ex is not unifiable.

Unification algorithm (cont.)

Example 8.9 $Ex_1 = \{Q(x, f(b)), Q(a, f(y))\}$.

- 1: $k = 0, \Theta_0 = \varepsilon, E_0 = Ex_1$.
- 2: $DS(E_0) = \{x, a\}, \Theta_1 = \{x \leftarrow a\}$.
- 3: $E_1 = \{Q(a, f(b)), Q(a, f(y))\}, k = 1$.
- 4: $DS(E_1) = \{b, y\}, \Theta_2 = \{x \leftarrow a, y \leftarrow b\}$.
- 5: $E_2 = \{Q(a, f(b))\}$.

Ex_1 is unifiable and $MGU(Ex_1) = \{x \leftarrow a, y \leftarrow b\}$.

Unification algorithm (cont.)

Example 8.10 $Ex_2 = \{Q(x, x), Q(a, b)\}$.

- 1: $k = 0, \Theta_0 = \varepsilon, E_0 = Ex_2$.
- 2: $DS(E_0) = \{x, a\}, \Theta_1 = \{x \leftarrow a\}$.
- 3: $E_1 = \{Q(a, a), Q(a, b)\}, k = 2$.
- 4: $DS(E_1) = \{a, b\}$.

Ex_2 is **NOT** unifiable!

Binary resolvents

For a literal λ , $\bar{\lambda}$ stands for the atom α corresponding to λ , that is

- if λ is an atom α , then $\bar{\lambda} = \alpha$
- if λ is the negation of an atom α (i.e. $\lambda = \neg\alpha$), then $\bar{\lambda} = \alpha$.

Let

- γ_1, γ_2 be clauses with **different** variables
- γ_1 contains a literal λ_1 and γ_2 contains a literal λ_2 such that λ_1 and λ_2 are complementary
- $\Theta = MGU(\bar{\lambda}_1, \bar{\lambda}_2)$.

A **binary resolvent** of γ_1 and γ_2 is a clause

$$ResBin(\gamma_1, \gamma_2) = (\gamma_1\Theta \setminus \lambda_1\Theta) \vee (\gamma_2\Theta \setminus \lambda_2\Theta)$$

Observe first that the clauses

$$\begin{aligned} P(x) \vee P(y) \\ \neg P(z) \vee \neg P(u) \end{aligned}$$

are inconsistent. However, by successively applying RR we cannot derive $\square$!

Factors

Observe first that the clauses

$$\begin{aligned} P(x) \vee P(y) \\ \neg P(z) \vee \neg P(u) \end{aligned}$$

are inconsistent. However, by successively applying RR we cannot derive $\square$!

To solve this problem, a *factorization process* is performed.

Factorization

- Let $\gamma = \lambda_1 \wedge \dots \wedge \lambda_n$, which can be viewed as a set $\{\lambda_1, \dots, \lambda_n\}$ of its literals.
- Take a unifiable subset $L \subseteq \{\lambda_1, \dots, \lambda_n\}$ of literals from γ_1 .
Put $\Theta = MGU(L)$.
- A **factor** of γ is a clause γ' obtained from $\gamma\Theta$ by removing repeating literals.

Factors (cont.)

Example 8.11 Consider a clause $\gamma_1 = P(x) \vee P(f(z)) \vee \neg Q(x)$.

- Take $L = \{P(x), P(f(z))\}$
- $\Theta = MGU(L) = \{x \leftarrow f(z)\}$
- $Factor(\gamma_1) = P(f(z)) \vee \neg Q(f(z))$.

Factors (cont.)

Example 8.11 Consider a clause $\gamma_1 = P(x) \vee P(f(z)) \vee \neg Q(x)$.

- Take $L = \{P(x), P(f(z))\}$
- $\Theta = MGU(L) = \{x \leftarrow f(z)\}$
- $Factor(\gamma_1) = P(f(z)) \vee \neg Q(f(z))$.

Let $\gamma_2 = \neg P(y, a) \vee \neg P(y, x) \vee \neg P(x, y)$.

- Take $L = \{\neg P(y, a), \neg P(y, x), \neg P(x, y)\}$
- $\Theta = MGU(L) = \{x \leftarrow a, y \leftarrow b\}$
- $Factor(\gamma_2) = \neg P(a, a)$ (**unit factor**).

Resolvents

A **resolvent** of two clauses γ_1 and γ_2 , in symbols $Res(\gamma_1, \gamma_2)$, is any of the following binary resolvent

- $ResBin(\gamma_1, \gamma_2)$
- $ResBin(\gamma_1, Factor(\gamma_2))$
- $ResBin(Factor(\gamma_1), \gamma_2)$
- $ResBin(Factor(\gamma_1), Factor(\gamma_2))$.

Resolution proof

A **resolution proof** of β from T is a sequence $(\gamma_0, \dots, \gamma_n)$ of clauses such that

- $\gamma_0 \in CL(T, \neg\beta)$
- $\gamma_n = \square$
- for every $i = 1, \dots, n$, either $\gamma_i \in CL(T, \neg\beta)$ or $\gamma_i = Res(\gamma_k, \gamma_j)$, for $0 \leq k, j < i$.

Resolution proof

A **resolution proof** of β from T is a sequence $(\gamma_0, \dots, \gamma_n)$ of clauses such that

- $\gamma_0 \in CL(T, \neg\beta)$
- $\gamma_n = \square$
- for every $i = 1, \dots, n$, either $\gamma_i \in CL(T, \neg\beta)$ or $\gamma_i = Res(\gamma_k, \gamma_j)$, for $0 \leq k, j < i$.

Theorem 8.6 (Soundness and Correctness) $T \vdash \beta$ iff there is a resolution proof of β from T .

Resolution method – outline

Given T and β , show $T \vdash \beta$.

Step 1: For every $\alpha \in T$ and for $\neg\beta$ determine its PCNF.

Step 2: If necessary, apply Skolemization. Get $CL = CL(T, \neg\beta)$.

Step 3: If $\square \in CL$, then STOP — returns **YES**.

Step 4: For any clauses $\gamma_1, \gamma_2 \in CL(T, \neg\beta)$, try to find unifiable literals $\lambda_1 \in \gamma_1$ and $\lambda_2 \in \gamma_2$ such that $\lambda_1 \Leftrightarrow \neg\lambda_2$. If no such clauses can be found, then STOP — return **NO**.

Step 5: Calculate $\gamma = Res(\gamma_1, \gamma_2)$.

Step 6: Put $CL := CL \cup \{\gamma\}$ and return to **Step 3**.

Resolution method – outline

Given T and β , show $T \vdash \beta$.

Step 1: For every $\alpha \in T$ and for $\neg\beta$ determine its PCNF.

Step 2: If necessary, apply Skolemization. Get $CL = CL(T, \neg\beta)$.

Step 3: If $\square \in CL$, then STOP — returns **YES**.

Step 4: For any clauses $\gamma_1, \gamma_2 \in CL(T, \neg\beta)$, try to find unifiable literals $\lambda_1 \in \gamma_1$ and $\lambda_2 \in \gamma_2$ such that $\lambda_1 \Leftrightarrow \neg\lambda_2$. If no such clauses can be found, then STOP — return **NO**.

Step 5: Calculate $\gamma = Res(\gamma_1, \gamma_2)$.

Step 6: Put $CL := CL \cup \{\gamma\}$ and return to **Step 3**.

Remark: This process need not be finite!

Answer Extraction

In many cases the query is of the form $\beta = \exists x. \phi(x)$ and we are actually interested in the question: “**Who has the property ϕ** ”. Obtaining answers for such queries is called an **answer extraction**.

General idea:

Replace a query $\beta = \exists x. \phi(x)$ by

$$\beta' = \exists x. \phi(x) \wedge \neg \textit{Answer}(x)$$

where *Answer* is a new predicate occurring nowhere else. Since it does not occur elsewhere, we cannot derive $\square$. Instead, we terminate the derivation process as soon as a clause containing only the answer predicate is produced.

Example 8.12a

Consider the set T of the following sentences:

$$\alpha_1 = \textit{Bird}(\textit{Tweety})$$

$$\alpha_2 = \forall x. \textit{Bird}(x) \rightarrow \textit{Flies}(x)$$

$$\alpha_3 = \forall x. \textit{Flies}(x) \rightarrow \textit{HasWings}(x)$$

and the formula $\beta = \textit{HasWings}(\textit{tweety})$.

Example 8.12a

Consider the set T of the following sentences:

$$\alpha_1 = \textit{Bird}(\textit{Tweety})$$

$$\alpha_2 = \forall x. \textit{Bird}(x) \rightarrow \textit{Flies}(x)$$

$$\alpha_3 = \forall x. \textit{Flies}(x) \rightarrow \textit{HasWings}(x)$$

and the formula $\beta = \textit{HasWings}(\textit{tweety})$.

The corresponding set $CL(T, \neg\beta)$ consists of the following clauses:

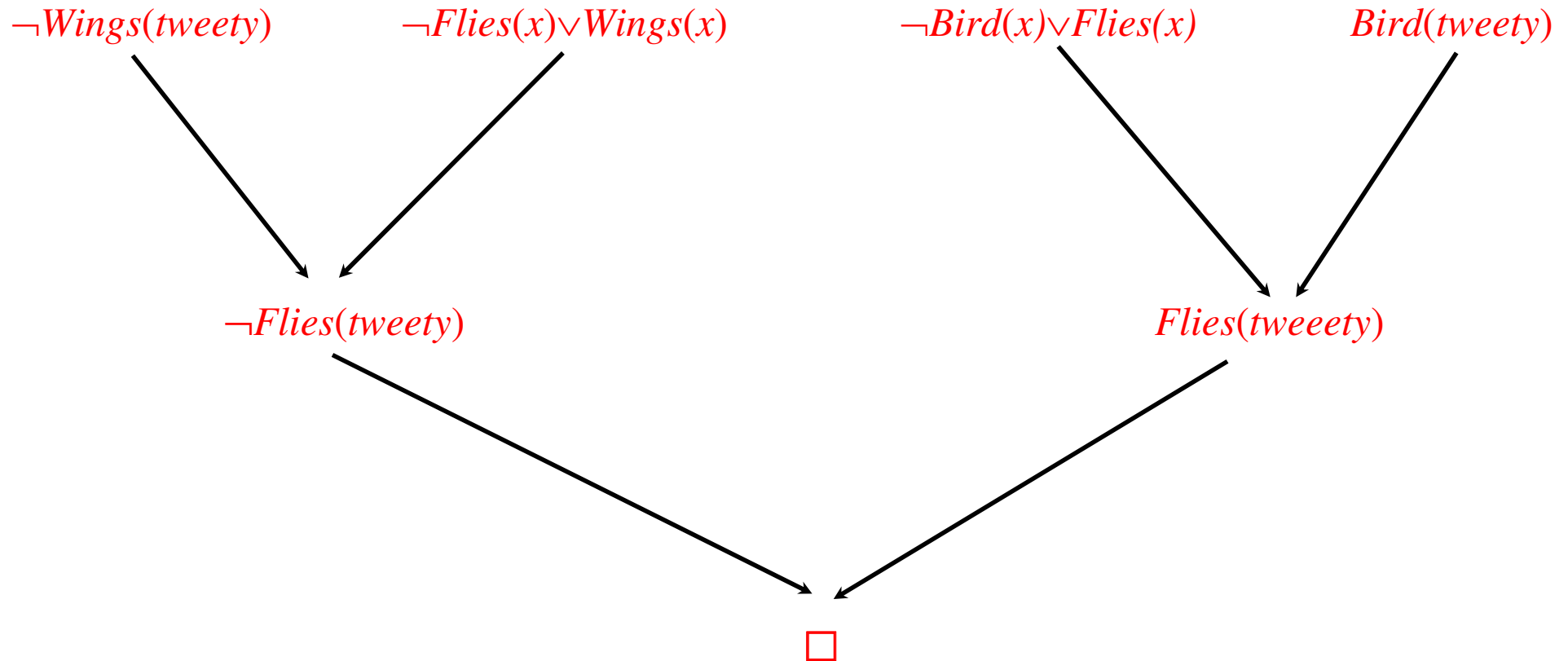
$$\textit{Bird}(\textit{Tweety})$$

$$\neg\textit{Bird}(x) \vee \textit{Flies}(x)$$

$$\neg\textit{Flies}(x) \vee \textit{HasWings}(x)$$

$$\neg\textit{HasWings}.$$

Example 8.12a: Derivation tree



Example 8.12b

Consider the same set T of sentences:

$$\textit{Bird}(\textit{Tweety})$$

$$\forall x. \textit{Bird}(x) \rightarrow \textit{Flies}(x)$$

$$\forall x. \textit{Flies}(x) \rightarrow \textit{HasWings}(x)$$

and the formula $\beta = \exists x. \textit{HasWings}(x)$.

Example 8.12b

Consider the same set T of sentences:

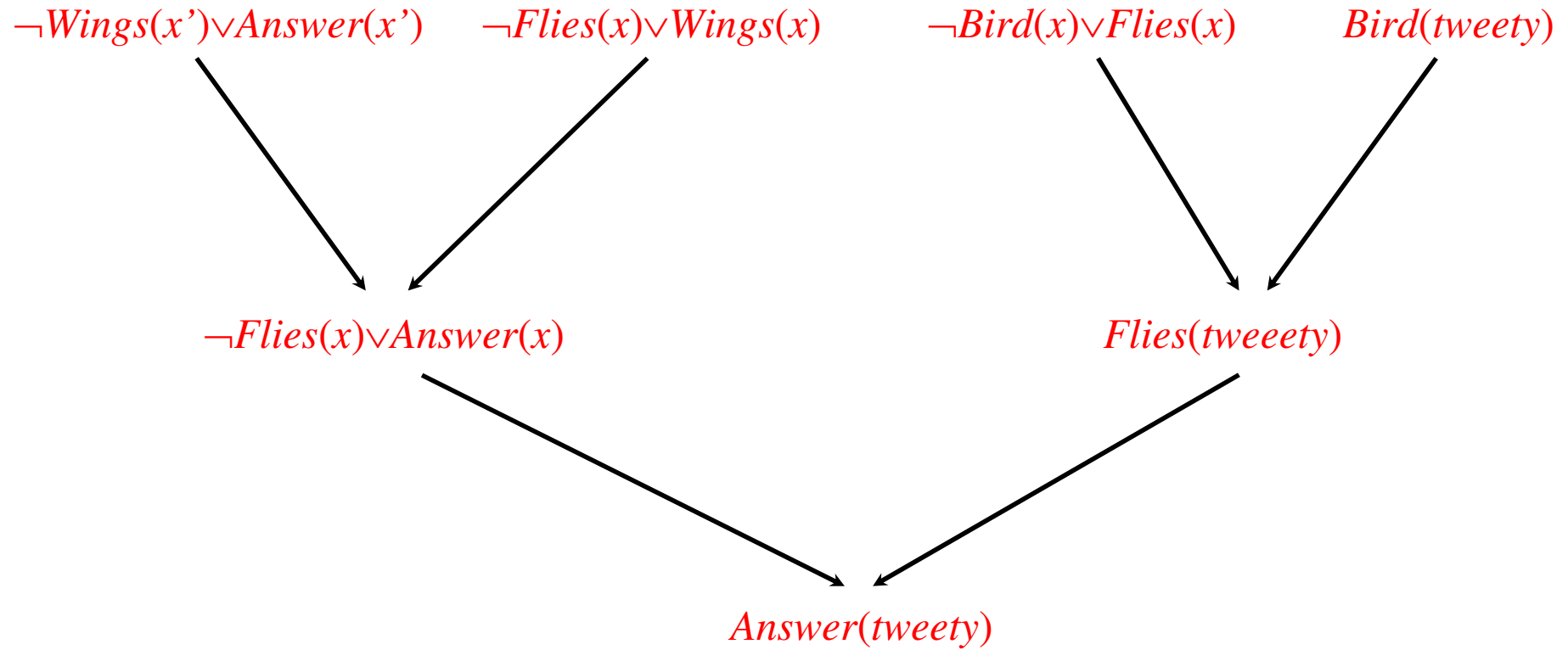
$$\textit{Bird}(\textit{Tweety})$$
$$\forall x. \textit{Bird}(x) \rightarrow \textit{Flies}(x)$$
$$\forall x. \textit{Flies}(x) \rightarrow \textit{HasWings}(x)$$

and the formula $\beta = \exists x. \textit{HasWings}(x)$.

The set $CL(T, \neg\beta)$ consists of clauses:

$$\textit{Bird}(\textit{Tweety})$$
$$\neg\textit{Bird}(x) \vee \textit{Flies}(x)$$
$$\neg\textit{Flies}(x) \vee \textit{HasWings}(x)$$
$$\neg\textit{HasWings}(x) \vee \textit{Answer}(x).$$

Example 8.12b: Derivation tree



Example 8.12c

Now let T consists of sentences:

$$Bird(Tweety) \vee Bird(Clyde)$$

$$\forall x. Bird(x) \rightarrow Flies(x)$$

$$\forall x. Flies(x) \rightarrow HasWings(x)$$

and let $\beta = \exists x. HasWings(x)$.

Example 8.12c

Now let T consists of sentences:

$$\begin{aligned} & Bird(Tweety) \vee Bird(Clyde) \\ & \forall x. Bird(x) \rightarrow Flies(x) \\ & \forall x. Flies(x) \rightarrow HasWings(x) \end{aligned}$$

and let $\beta = \exists x. HasWings(x)$.

Intuitively, T implies that *Tweety* has wings, or *Clyde* has wings, or both have wings, i.e. $T \vdash HasWing(Tweety) \vee HasWings(Clyde)$.

Example 8.12c

Now let T consists of sentences:

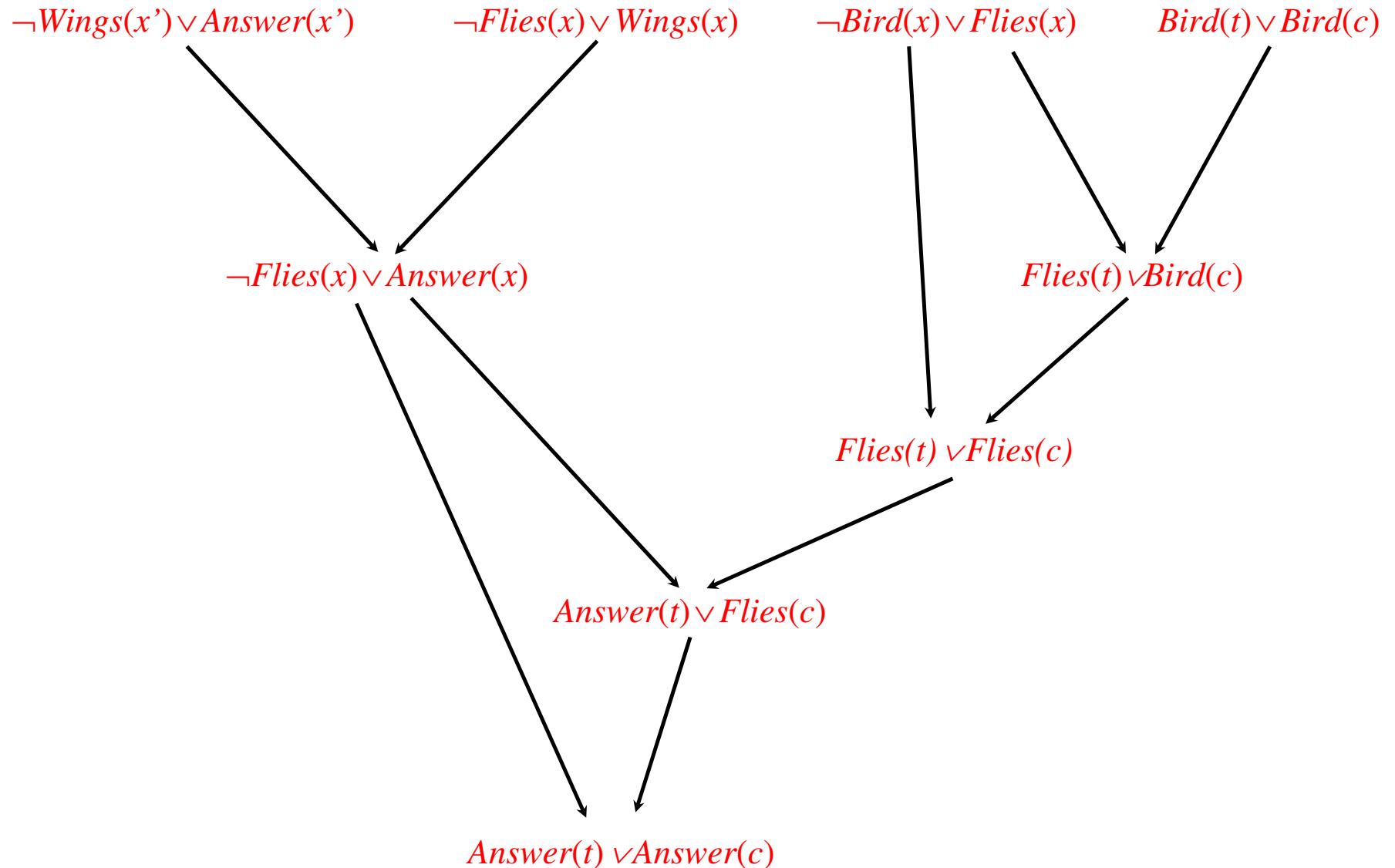
$$\begin{aligned} & Bird(Tweety) \vee Bird(Clyde) \\ & \forall x. Bird(x) \rightarrow Flies(x) \\ & \forall x. Flies(x) \rightarrow HasWings(x) \end{aligned}$$

and let $\beta = \exists x. HasWings(x)$.

Intuitively, T implies that *Tweety* has wings, or *Clyde* has wings, or both have wings, i.e. $T \vdash HasWing(Tweety) \vee HasWings(Clyde)$.

The set $CL(T, \neg\beta)$ is as before.

Example 8.12c: Derivation tree



Thank you for your attention!

Any questions are welcome.